

EC-341 Digital Systems Design

Experiment # 7

Analysis of Implementation of Combinational Circuits Using Different Approaches

(Design ALU)

Objectives:

- Combinational Circuits
- Implementation using ALU
- Comparison of Area Estimations
- Lab Tasks

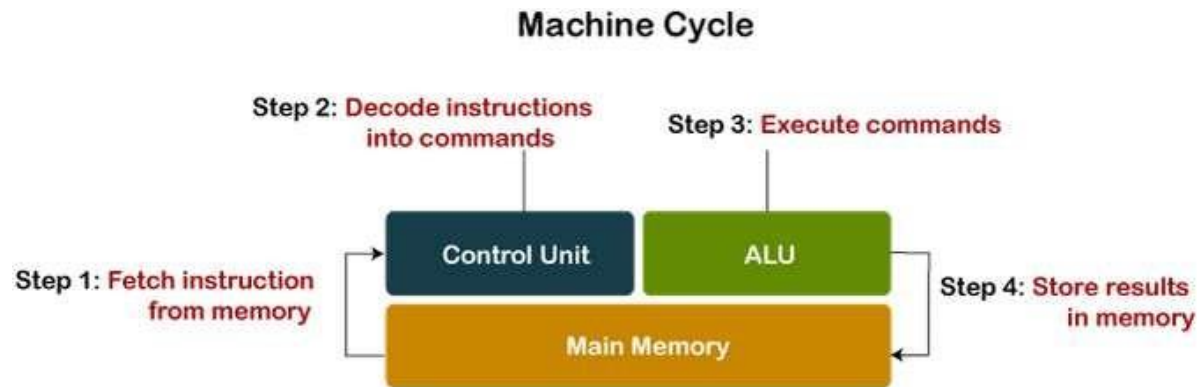
1. Combinational Circuits

Combinational Circuits (CC) are circuits made up of different types of logic gates. A logic gate is a basic building block of any electronic circuit. The output of the combinational circuit depends on the values at the input at any given time. The circuits do not make use of any memory or storage device. Some of the most common types of combinational circuit include adder, subtractor, multiplexer and de-multiplexer.

2. Arithmetic Logic Unit

In the computer system, ALU is a main component of the central processing unit, which stands for arithmetic logic unit and performs arithmetic and logic operations. It is also known as an integer unit (IU) that is an integrated circuit within a CPU or GPU, which is the last component to perform calculations in the processor. It has the ability to perform all processes related to arithmetic and logic operations such as addition, subtraction, and shifting operations, including Boolean comparisons (XOR, OR, AND, and NOT operations). Also, binary numbers can accomplish mathematical and bitwise operations.

The arithmetic logic unit is split into AU (arithmetic unit) and LU (logic unit). The operands and code used by the ALU tell it which operations have to perform according to input data. When the ALU completes the processing of input, the information is sent to the computer's memory.



Except performing calculations related to addition and subtraction, ALUs handle the multiplication of two integers as they are designed to execute integer calculations; hence, its result is also an integer. However, division operations commonly may not be performed by ALU as division operations may produce a result in a floating-point number. Instead, the floating-point unit (FPU) usually handles the division operations; other non-integer calculations can also be performed by FPU.

Additionally, engineers can design the ALU to perform any type of operation. However, ALU becomes more costly as the operations become more complex because ALU destroys more heat and takes up more space in the CPU. This is the reason to make powerful ALU by engineers, which provides the surety that the CPU is fast and powerful as well.

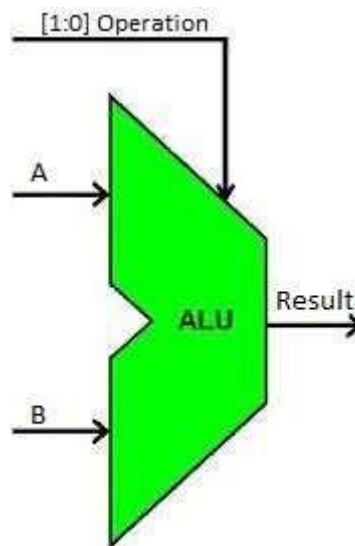
The calculations needed by the CPU are handled by the arithmetic logic unit (ALU); most of the operations among them are logical in nature. If the CPU is made more powerful, which is made on the basis of the ALU is designed. Then it creates more heat and takes more power or energy. Therefore, it must be moderation between how complex and powerful ALU is and not be more costly. This is the main reason the faster CPUs are more costly; hence, they take much power and destroy more heat. Arithmetic and logic operations are the main operations that are performed by the ALU; it also performs bit-shifting operations.

Although the ALU is a major component in the processor, the ALU's design and function may be different in the different processors. For case, some ALUs are designed to perform only integer calculations, and some are for floating-point operations. Some processors include a single arithmetic logic unit to perform operations, and others may contain numerous ALUs to complete calculations. The common operations performed by ALU are:

- **Addition.** Adds A and B with carry-in or carry-out sum at Y.
- **Subtraction.** Subtracts B from A or vice versa with the difference at Y and carry-in or carry-out.
- **Increment.** Where A or B is increased by one and Y represents the new value.

- **Decrement.** Where A or B is decreased by one and Y represents the new value.
- **AND.** The bitwise logic AND of A and B is represented by Y.
- **OR.** The bitwise logic OR of A and B is represented by Y.
- **Exclusive-OR.** The bitwise logic XOR of A and B is represented by Y.

Block Diagram



Verilog Design for One Bit ALU

```
module ALUcode(a, b, operation, Result);
    input a, b;
    input [1:0] operation;
    output Result;
    reg Result;

    always@(a or b or operation)
    begin
        case (operation)
            2'b00: Result = a & b;    // AND
            2'b01: Result = a | b;    // OR
            2'b10: Result = a + b;    // addition
            2'b11: Result = a - b;    // subtraction
        endcase
    end
endmodule
```

```

module testBench(

);
reg A,B;
reg[1:0] ALU_Sel;

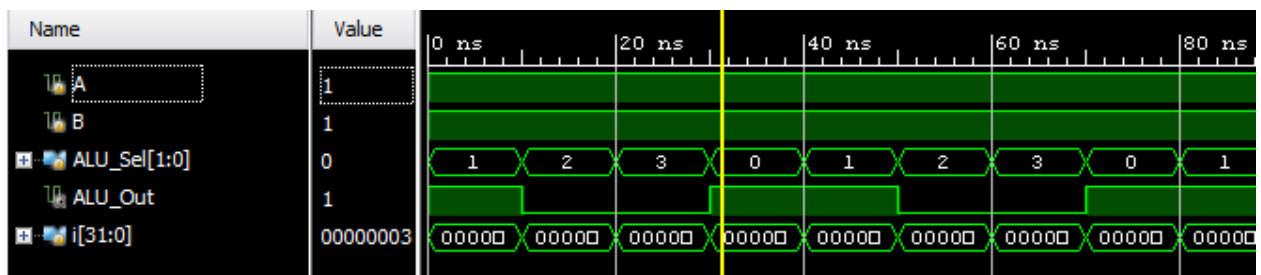
//Outputs
wire ALU_Out;
integer i;
ALUcode test_unit(A,B,ALU_Sel,ALU_Out);
initial
begin
    A = 1;
    B = 1;
    ALU_Sel = 2'b00;

    for (i=0;i<=15;i=i+1)
    begin
        ALU_Sel = ALU_Sel + 2'b01;
        #10;
    end;

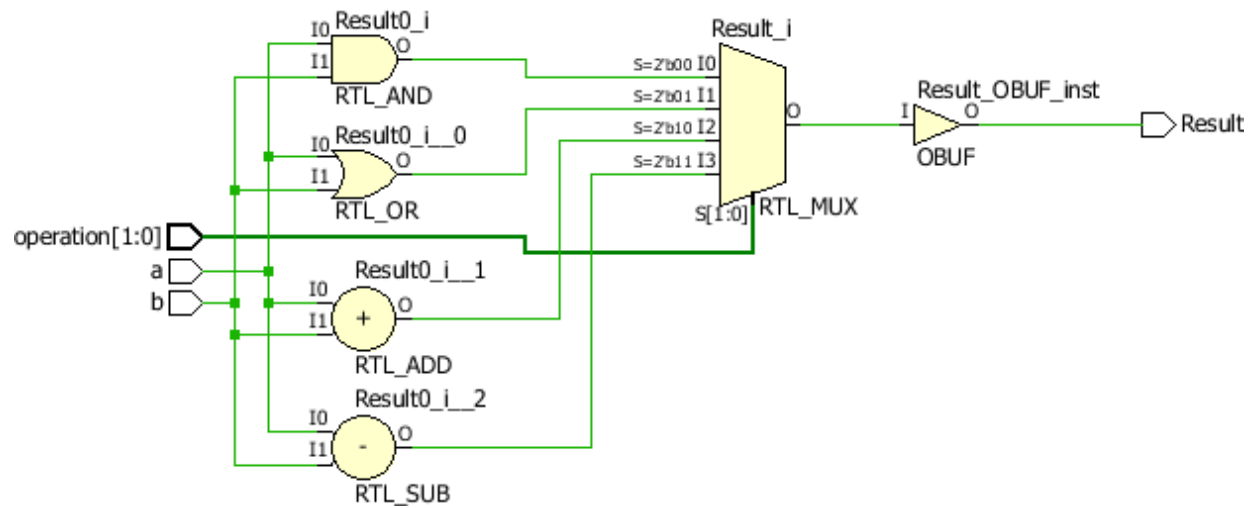
end
endmodule

```

Output



Schematic: -



Analysis: -

Analysis based on Area Comparison.

Area: -

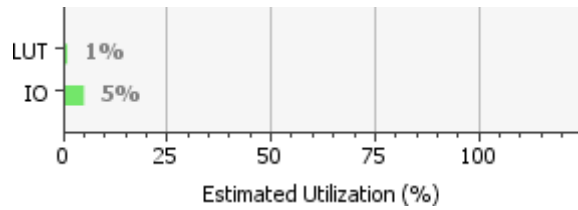
Report Cell Usage:

	Cell	Count
1	LUT4	1
2	IBUF	4
3	OBUF	1

Report Instance Areas:

	Instance	Module	Cells
1	top		6

Graph: -

**Resource Utilization: -**

Resource	Estimation	Available	Utilization %
LUT	1	17600	0.01
IO	5	100	5.00

3. Tasks

1. Write a Gate Level Verilog Code for Basic ALU. Verify results using different instances. Analyze time & size estimations. Draw Graphs to show differences.
 2. Write a Gate Level and Behavioral Level Verilog Code for two Bit ALU which supports three bit Operations. Verify results using different instances. Analyze area estimations. Draw Graphs to show differences.
-